

Terraform Study Notes

Table of Contents

[Why Terraform?](#)

[Terraform Providers](#)

[Downloading Providers](#)

[Terraform Initialization \(init\)](#)

[Defining Providers in Code](#)

[Essential Terraform Commands](#)

[terraform fmt](#)

[terraform validate](#)

[terraform plan](#)

[terraform apply](#)

[Resources and Dependencies](#)

[Resource Definition Example](#)

[Resource Addressing](#)

[Implicit Dependencies](#)

[Explicit Dependencies](#)

[Terraform State and Lifecycle](#)

[State File Concepts](#)

[State Locking and Unlocking](#)

[Enabling Blob Versioning in Azure](#)

[Variables](#)

[Variable Types](#)

[Declaration and Assignment](#)

[State File Scenarios](#)

[Managing State Drift](#)

Why Terraform?

- Infrastructure as Code (IaC) tool.
- Uses blocks and arguments for configuration.
- Provision infrastructure across multiple cloud providers.
- Uses HashiCorp Configuration Language (HCL).
- Standard file extension is **.tf**.

- Has its own centralized repository: Terraform Registry.

Terraform Providers

- Providers include Azure, AWS, and GCP.
- Each cloud has its own specific provider.
- Acts as a plugin to connect Terraform to specific cloud APIs.

Downloading Providers

- Available via the [Terraform Registry](#).
- Triggered by the **terraform init** command.

Terraform Initialization (init)

- Initializes the environment and plugins.
- Downloads providers into the **.terraform** folder.
- On Windows, provider binaries are **.exe** files.
- Version locking is managed via the **.terraform.lock.hcl** file.

Defining Providers in Code

Providers are defined within the **terraform** block using the **required_providers** block. You must specify the provider name, source, and version.

None

```
terraform {  
  required_providers {  
    azurerm = {  
      source  = "hashicorp/azurerm"  
      version = "5.2.0"  
    }  
  }  
}
```

Essential Terraform Commands

terraform fmt

- Formats HCL files to follow a standard style.
- Beautifies code to improve readability.

terraform validate

- Checks for syntax errors and internal consistency.
- Used to maintain code quality.

terraform plan

- Requires cloud provider authentication (e.g., Azure login with Contributor access).
- Provides a preview of changes: additions, updates, or deletions.
- Compares the state file with the actual infrastructure (detects "drift").
- Does not make actual changes to the cloud environment.

terraform apply

- Executes the plan to create/update infrastructure.
- Shows the plan and asks for manual confirmation (type "yes").
- Use **--auto-approve** to skip manual confirmation.

Resources and Dependencies

Resource Definition Example

None

```
resource "azurerm_resource_group" "example" {  
  name      = "SSK-RG-001"  
  location = "West Europe"  
}
```

Resource Addressing

Format: <Type>.<Name>.<Argument>

Example: **azurerm_resource_group.example.name**

Implicit Dependencies

Occurs when one resource references an attribute of another resource. Terraform automatically determines the creation order.

None

```
resource "azurerm_resource_group" "example" {
  name      = "SSK-RG-001"
  location  = "West Europe"
}

resource "azurerm_storage_account" "sukhi" {
  name                        = "storageaccountname"
  resource_group_name        = azurerm_resource_group.example.name
  location                   =
azurerm_resource_group.example.location
  account_tier               = "Standard"
  account_replication_type   = "GRS"
}
```

Explicit Dependencies

Defined using the **depends_on** meta-argument when a dependency is not automatically visible to Terraform.

None

```
resource "azurerm_storage_account" "example" {
  name                        = "storageaccountname"
  # ... other arguments
  depends_on = [azurerm_resource_group.example]
}
```

Terraform State and Lifecycle

State File Concepts

- Acts as a "diary" or "memory" of the infrastructure.
- Stores current metadata about managed resources.
- Essential for team collaboration and detecting manual changes (drift).

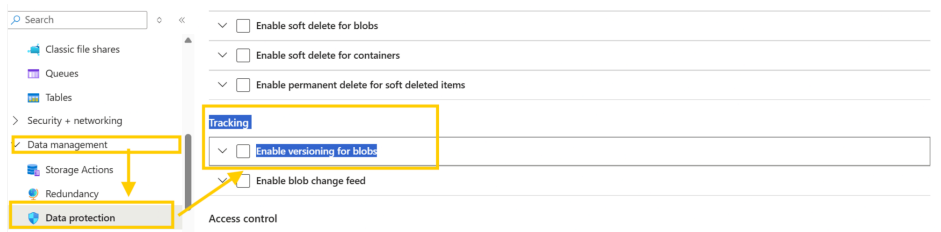
State Locking and Unlocking

Terraform locks the state file during **apply** to prevent concurrent changes. If a lock is stuck, it can be broken via:

- **CLI:** terraform force-unlock <LOCK_ID>
- **Azure Portal:** Storage Account > Blob Containers > Break Lease.

Enabling Blob Versioning in Azure

1. Login to Azure Portal.
2. Navigate to the **Storage Account**.
3. Go to **Data Management > Data Protection**.
4. Under the "Tracking" section, check **Enable versioning for blobs**.



Variables

Variables act as placeholders to avoid hardcoding values and improve code reusability.

Variable Types

- **string:** Text in double quotes.
- **number:** Numeric values.
- **bool:** true/false.
- **list:** Collection of values (supports duplicates).
- **map:** Key-value pairs.
- **object:** Complex structural types.

Declaration and Assignment

Declaration:

```
None
variable "instance_name" {
  type    = string
  default = "production-server"
}
```

Assignment:

Values are typically assigned in **.tfvars** or **.auto.tfvars** files.

Usage:

Reference them in code using: **var.variable_name**

State File Scenarios

1. **Resource deleted from config (.tf file)**
 - Results in deletion of resource from state file & cloud.
2. **Resource deleted from cloud, which is managed by terraform**
 - Results in terraform trying to add that resource in cloud.
3. **Resource added/update into config (.tf file)**
 - Results in terraform trying to add/modify resource in cloud.
4. **Modify Resource Manually in Cloud, which is managed by terraform**
 - DRIFT (something changed manually in cloud that is managed by terraform).
5. **Resource added in cloud, needs to be managed by terraform**

Terraform import command / import block format and add.

Managing State Drift

What is Drift?

- Drift is the difference between desired state and current state.
- Drift is when Azure resources were changed manually which are managed by Terraform.

How to manage state drift?

- Check Terraform plan output.
- Find the drift.

What to do when drift is found?

- Check with the change maker: Are changes needed or temporary (and can be reverted)?

How to check who made the changes?

5. Go to the Azure portal.
6. Navigate to the resource having a drift.
7. Go to activity logs.

Case 1: Manual changes need to be kept in cloud

- Make the change in Terraform config files.
- Verify Terraform plan.
- Run Terraform apply.

Case 2: Manual changes not needed in cloud can be reverted

Run Terraform apply to revert those changes.